Social Network Simulation (COL106 Long Assignment 2)

Implementation of a miniature social network using custom data structures — Graph for friendships and AVL Tree for user posts.

1. Team / Author Information

- **Name:** Tanmay Modi
- **Course:** COL106 – Data Structures and Algorithms
- **Instructors:** Rohit Vaish, Rajendra Kumar

2. Overview

This project implements a simplified social network system supporting user management, friendships, posts, and queries. All core functionality is built using custom data structures, without relying on STL containers such as `std::set` or `std::map`.

The program supports:

- Adding users and friendships
- Posting messages
- Suggesting friends
- Computing degrees of separation
- Listing posts in reverse chronological order

3. Data Structures Used

3.1. Graph

- Nodes represent users.
- Edges represent friendships.
- Implemented via adjacency lists.
- Supports BFS for shortest path computation (degrees of separation).

3.2. AVL Tree

- Stores each user's posts.
- Sorted by timestamp, and lexicographically when timestamps tie.
- Self-balancing using AVL rotations on insertions.

4. Supported Commands

Social Network Operations

- **ADD_USER <username>**
  Adds a new user to the network, initially with no friends and no posts.

- **ADD_FRIEND <username1><username2>**
  Establishes a bidirectional friendship between two existing users.
- **LIST_FRIENDS <username>**
  Prints an alphabetically sorted list of the specified user's friends.
- **SUGGEST_FRIENDS <username><N>**
  Recommends up to N new friends who are "friends of a friend" but not already friends.
  - **Ranking:** Candidates are ranked by the number of mutual friends (descending).
  - **Ties:** Broken alphabetically by username.
  - **Edge Cases:** If N = 0, output nothing. If fewer than N candidates exist, list all available.
- **DEGREES_OF_SEPARATION <username1><username2>**
  Calculates the length of the shortest friendship path between two users. If no path exists, outputs `-1`.

User Content Operations

- **ADD_POST <username><post content>**
  Adds a new post with the specified content to the user's posts.
- **OUTPUT_POSTS <username><N>**
  Displays the most recent N posts of the user in reverse chronological order.
  If N = `-1`, output all posts. If the user has fewer than N posts, output all available.

Note: Usernames and post contents are case-insensitive (e.g., `Tanmay` and `tanmay` are considered identical).

5. Input and Output Format

Input

A list of commands, one per line.

Output

Responses printed to `stdout` according to command results. Invalid or duplicate operations produce error messages such as:

- Invalid Username
- Invalid post count
- Username already exists

Example Input

```
ADD_USER alice
ADD_USER bob
ADD_FRIEND alice bob
ADD_POST alice Hello world!
OUTPUT_POSTS alice 1
EXIT
```

Example Output

```
Hello world!
```

6. Compilation and Execution

Use the provided `compile.sh` or `run.bat`.

7. Files Included

- main.cpp
- header.hpp
- graph.cpp/hpp
- avl_tree.cpp/hpp
- socialnet.cpp/hpp
- user.hpp
- compile.sh / run.bat
- README.md

8. Assumptions and Design Decisions

- Usernames are case-insensitive and unique.
- All commands are single-line. Non-single line commands are not valid.
- Adding an existing user prints `Username already exists.`
- Friendships are mutual and cannot be self-links. Trying to do otherwise prints `Friendships must be between different users`.
- Posts are stored per user, sorted by timestamp descending.
- No STL containers used for main graph/tree logic.
- Using invalid commands prints `Unknown command`.
- Invalid usernames are handled by printing `Username not found` .
- Trying to get posts for users with no posts prints `No posts found for user`.
- Degrees of Separation for two disconnected users prints `-1`.
- Giving an invalid post count prints `Invalid post count`.

9. Sample Test Coverage

The implementation has been tested on:

- Duplicate user additions
- Invalid friend requests
- Edge cases (e.g., `OUTPUT_POSTS username -1`)
- Stress tests with over 1000 operations

10. Limitations

- Friend suggestions limited to two-hop connections.
- Time collisions handled lexicographically.

11. Acknowledgments

Assignment specification provided by the Department of Computer Science and Engineering, IIT Delhi.